

Procesamiento de datos estructurados

M9: Fundamentos de big data

|AE4: Manipular grandes volúmenes de datos utilizando spark sql para el procesamiento de grandes volúmenes de datos estructurados.

Introducción

En los proyectos de análisis de datos, gran parte de la información que se procesa se encuentra en forma estructurada, es decir, organizada en tablas, con filas y columnas que representan registros y atributos. Este tipo de datos es habitual en bases de datos relacionales, hojas de cálculo, archivos CSV, JSON y Parquet, entre otros. Gestionar, consultar y transformar estos datos de forma eficiente es un desafío esencial en entornos de Big Data.

Apache Spark no solo ofrece capacidades avanzadas para el procesamiento distribuido de datos no estructurados, sino que también proporciona un potente módulo llamado **Spark SQL**, diseñado específicamente para trabajar con datos estructurados y semiestructurados. Spark SQL amplía las capacidades de Spark, permitiendo la consulta de datos mediante sentencias SQL, la manipulación de **DataFrames** y la integración con diversas fuentes de datos.

Este manual está orientado a que el lector comprenda el funcionamiento de Spark SQL, las ventajas de utilizar DataFrames y las técnicas de optimización que permiten acelerar el procesamiento de consultas. Además, se abordará la interoperabilidad entre RDD y DataFrames, así como el uso de formatos de almacenamiento eficientes como **JSON** y **Parquet**.

El procesamiento de datos estructurados con Spark representa un recurso clave para analistas de datos, ingenieros y científicos de datos que necesitan manejar grandes volúmenes de información estructurada, integrando consultas SQL tradicionales con la potencia del procesamiento distribuido.

Aprendizaje esperado

Cuando finalices la lección serás capaz de:

- Comprender qué es Spark SQL y sus principales características.
- Explicar qué es una sesión Spark y cómo se configura.
- Definir qué es un DataFrame en Spark y cómo funciona.
- Conocer los métodos de creación y exploración de DataFrames.
- Utilizar DataFrames para realizar consultas sobre datos estructurados.
- Comprender las técnicas de optimización de consultas (Catalyst y Tungsten).
- Distinguir entre DataFrame y Dataset y sus principales diferencias.
- Aplicar funciones definidas por el usuario (UDFs) en Spark SQL.

Procesamiento de Datos Estructurados

1. Spark SQL: Qué es Spark SQL

Spark SQL es un módulo de **Apache Spark** diseñado para trabajar con datos estructurados y semiestructurados. Ofrece una interfaz que permite realizar consultas utilizando el lenguaje SQL estándar, así como operaciones programáticas sobre estructuras de datos distribuidas llamadas **DataFrames** y **Datasets**. Este módulo combina la familiaridad del lenguaje SQL con la potencia del procesamiento distribuido de Spark.

La incorporación de Spark SQL en el ecosistema Spark respondió a la necesidad de procesar datos estructurados de manera más eficiente y sencilla. Anteriormente, Spark se centraba en el uso de **RDDs (Resilient Distributed Datasets)**, que ofrecían un alto nivel de flexibilidad pero requerían escribir un código más complejo para realizar consultas estructuradas. Con Spark SQL, se simplificó el acceso y manipulación de estos datos.

Uno de los principales beneficios de Spark SQL es que permite ejecutar consultas SQL tradicionales sobre grandes conjuntos de datos distribuidos, sin necesidad de mover los datos a un sistema gestor de bases de datos relacional (RDBMS). Esto facilita la integración de Spark en flujos de trabajo existentes que utilizan SQL.

Además, Spark SQL proporciona una **API unificada** que permite a los usuarios trabajar con datos utilizando SQL, DataFrames y Datasets de forma intercambiable. Esto significa que los desarrolladores pueden combinar operaciones SQL con transformaciones funcionales en el mismo flujo de trabajo, aumentando la flexibilidad y productividad.

Spark SQL soporta múltiples fuentes de datos, incluyendo archivos **CSV**, **JSON**, **Parquet**, **ORC**, bases de datos JDBC y sistemas de almacenamiento distribuido como **HDFS**, **S3** y **Azure Blob Storage**. Esta capacidad permite integrar datos provenientes de diferentes plataformas en un mismo análisis.

Otro aspecto destacado de Spark SQL es su capacidad de optimización. Utiliza un motor de consulta llamado **Catalyst Optimizer**, que analiza y mejora los planes de ejecución de las consultas para maximizar la eficiencia. Asimismo,

Spark SQL aprovecha el proyecto **Tungsten** para optimizar el uso de la memoria y la CPU durante la ejecución.

En términos prácticos, Spark SQL es una herramienta poderosa para realizar tareas como la limpieza de datos, integración de fuentes diversas, generación de reportes y análisis exploratorio, todo a gran escala y con altos niveles de rendimiento.

En conclusión, Spark SQL amplía las capacidades de Apache Spark al permitir el procesamiento eficiente y flexible de datos estructurados, integrando la

2. Características de Spark SQL

Spark SQL se ha consolidado como uno de los módulos más utilizados de Apache Spark, gracias a sus características que combinan flexibilidad, rendimiento y facilidad de uso. Estas características permiten a los desarrolladores y analistas de datos trabajar con grandes volúmenes de datos estructurados de manera eficiente.

Una de las características más destacadas de Spark SQL es su **soporte para múltiples lenguajes de programación**. Los usuarios pueden escribir consultas y manipular datos utilizando **SQL, Python, Scala, Java y R**. Esto facilita la adopción de Spark SQL en equipos multidisciplinarios con conocimientos en diferentes lenguajes.

Otra característica importante es la posibilidad de ejecutar **consultas SQL tradicionales** sobre grandes volúmenes de datos. Spark SQL permite realizar consultas complejas utilizando la sintaxis estándar de SQL, lo que resulta familiar para analistas y profesionales acostumbrados a trabajar con bases de datos relacionales.

Spark SQL también destaca por su capacidad para trabajar con **DataFrames y Datasets**, estructuras de datos distribuidas que permiten manipular información estructurada de manera eficiente. Los DataFrames representan tablas con columnas y filas, similares a una hoja de cálculo o a una tabla en una base de datos.

Además, Spark SQL es capaz de integrarse con una amplia variedad de **fuentes de datos**, como archivos **CSV, JSON, Parquet, ORC**, bases de datos JDBC y sistemas de almacenamiento distribuido como **HDFS, Amazon S3 y Azure Blob**.

Storage. Esto facilita la combinación de datos provenientes de diferentes sistemas en un único análisis.

Una característica clave de Spark SQL es su **motor de optimización Catalyst**. Este motor analiza las consultas y genera planes de ejecución optimizados que minimizan el movimiento de datos y aprovechan al máximo los recursos del clúster. Esto se traduce en una mejora significativa del rendimiento de las consultas.

El módulo Spark SQL también incluye el proyecto **Tungsten**, que optimiza el uso de la memoria y la CPU mediante técnicas como la generación de código a bajo nivel y la gestión eficiente de la memoria. Gracias a estas optimizaciones, Spark SQL es capaz de superar a muchos sistemas tradicionales de procesamiento de datos.

Otra ventaja de Spark SQL es la posibilidad de realizar **consultas interactivas**. Los usuarios pueden conectarse a un clúster Spark y ejecutar consultas SQL en tiempo real, obteniendo resultados de manera rápida y eficiente.

3. Qué es una sesión Spark

En el contexto de **Apache Spark**, la **sesión Spark** (o **SparkSession**) es el punto de entrada principal para interactuar con las funcionalidades de Spark, especialmente con las relacionadas al módulo **Spark SQL** y al manejo de datos estructurados mediante **DataFrames** y **Datasets**.

Antes de la introducción de SparkSession, las aplicaciones Spark utilizaban un objeto llamado **SparkContext** para inicializar el entorno de ejecución. Sin embargo, desde la versión 2.0 de Spark, la clase SparkSession unificó y simplificó la creación y configuración de las aplicaciones, integrando tanto el SparkContext como el SQLContext en un solo objeto.

Una sesión Spark proporciona acceso a todas las capacidades de Spark SQL, incluyendo la lectura y escritura de datos, la creación de DataFrames y Datasets, la ejecución de consultas SQL y la gestión de configuraciones específicas de la aplicación.

La creación de una sesión Spark es sencilla y requiere especificar algunos parámetros básicos, como el nombre de la aplicación y el modo de ejecución (local o en clúster). Por ejemplo, en **PySpark**, se puede crear una sesión Spark de la siguiente manera:

```
from pyspark.sql import SparkSession

spark = SparkSession.builder \
    .appName("MiAplicacionSQL") \
    .master("local[*]") \
    .getOrCreate()
```

Una vez creada la sesión, es posible acceder al **SparkContext** y al **SQLContext** a través de sus atributos:

```
sc = spark.sparkContext
sqlContext = spark
```

La sesión Spark también permite configurar parámetros importantes, como la cantidad de memoria asignada a los ejecutores, la cantidad de núcleos utilizados y la ubicación de los archivos de entrada y salida.

Otra ventaja de la sesión Spark es que mantiene el estado de la aplicación durante toda su ejecución. Esto significa que los usuarios pueden ejecutar múltiples consultas y transformaciones sobre los datos sin necesidad de reiniciar el contexto de ejecución.

Es importante cerrar la sesión Spark al finalizar la aplicación para liberar los recursos asignados. Esto se realiza mediante el método `stop()`:

```
spark.stop()
```

En resumen, la sesión Spark es un componente esencial en cualquier aplicación desarrollada con Apache Spark. Proporciona un entorno unificado y flexible para trabajar con datos estructurados, facilitando la creación, manipulación y consulta de información a gran escala.

4. DataFrames: Qué es un DataFrame Spark y cómo funciona

Uno de los elementos clave en el procesamiento de datos estructurados con **Apache Spark** es el concepto de **DataFrame**. Un **DataFrame** es una colección

distribuida de datos organizados en columnas, similar a una tabla en una base de datos relacional o a un DataFrame en Python (Pandas) o R.

El DataFrame en Spark ofrece una abstracción de alto nivel que permite manipular y consultar datos estructurados de manera eficiente, utilizando tanto funciones de programación como consultas SQL. Esta estructura de datos fue introducida para simplificar el trabajo con datos tabulares y mejorar el rendimiento del procesamiento.

Internamente, un DataFrame se basa en un conjunto de RDDs, pero añade información estructural sobre las columnas y sus tipos de datos, lo que permite realizar optimizaciones adicionales mediante el motor **Catalyst**. Esta información estructurada facilita la ejecución de consultas complejas y permite aplicar técnicas de optimización que no son posibles con RDDs tradicionales.

Un DataFrame puede contener datos de cualquier tipo, como números, cadenas de texto, fechas y estructuras más complejas como arrays y mapas. Además, admite datos nulos y permite definir esquemas personalizados para describir la estructura de los datos.

El funcionamiento de un DataFrame se basa en el principio de **evaluación perezosa** (lazy evaluation). Esto significa que las operaciones sobre un DataFrame no se ejecutan inmediatamente, sino que se registran como un plan lógico de ejecución. Este plan se optimiza y ejecuta únicamente cuando se solicita una acción que requiera un resultado, como `show()` o `count()`.

Una de las principales ventajas de los DataFrames es su compatibilidad con múltiples lenguajes de programación, incluyendo **Python (PySpark)**, **Scala**, **Java** y **R**. Esto facilita su adopción por parte de equipos multidisciplinarios y permite integrar Spark en flujos de trabajo existentes.

Los DataFrames también permiten realizar operaciones avanzadas sobre los datos, como filtrado, agregación, ordenamiento, unión y combinación de conjuntos de datos, así como la ejecución de consultas SQL estándar. Estas operaciones pueden combinarse para construir flujos de trabajo complejos de análisis de datos.

Además, los DataFrames son interoperables con otros componentes de Spark, como Spark SQL, MLlib (biblioteca de aprendizaje automático) y GraphX (biblioteca de grafos). Esto permite integrar el procesamiento de datos estructurados en flujos de trabajo de análisis avanzados.

En conclusión, los DataFrames son una herramienta fundamental en Spark para trabajar con datos estructurados. Su diseño flexible y optimizado facilita el procesamiento eficiente de grandes volúmenes de información, integrando las ventajas de los RDDs con la simplicidad y familiaridad de las consultas SQL.

5. Métodos de creación de un DataFrame

La creación de un **DataFrame** en Apache Spark es un proceso flexible que puede realizarse a partir de diferentes fuentes de datos y estructuras existentes. Spark proporciona diversos métodos para crear DataFrames, adaptándose a distintos escenarios y necesidades de análisis.

Una de las formas más sencillas de crear un DataFrame es a partir de una colección de datos local, como una lista o un diccionario en Python. Por ejemplo, utilizando **PySpark**, se puede crear un DataFrame a partir de una lista de tuplas:

```
from pyspark.sql import SparkSession

spark = SparkSession.builder.appName("EjemploDataFrame").getOrCreate()

datos = [("Ana", 25), ("Luis", 30), ("Carla", 28)]
columnas = ["Nombre", "Edad"]

df = spark.createDataFrame(datos, columnas)
df.show()
```

Otra manera común de crear un DataFrame es mediante la lectura de archivos estructurados. Spark SQL soporta múltiples formatos de archivo, como **CSV**, **JSON**, **Parquet**, **ORC** y **Avro**. Por ejemplo, para leer un archivo CSV:

```
df_csv = spark.read.csv("ruta/datos.csv", header=True, inferSchema=True)
df_csv.show()
```

También es posible crear un DataFrame leyendo archivos en formato **JSON**:

```
df_json = spark.read.json("ruta/datos.json")
df_json.show()
```

El formato **Parquet**, que ofrece almacenamiento eficiente y optimizado para consultas analíticas, también puede ser utilizado para crear un DataFrame:


```
df_parquet = spark.read.parquet("ruta/datos.parquet")
df_parquet.show()
```

Otra opción para crear un DataFrame es utilizando fuentes de datos externas, como bases de datos relacionales. Spark SQL permite conectarse a bases de datos mediante el protocolo **JDBC**. Por ejemplo:

```
df_jdbc = spark.read \
    .format("jdbc") \
    .option("url", "jdbc:mysql://localhost:3306/basededatos") \
    .option("dbtable", "tabla") \
    .option("user", "usuario") \
    .option("password", "contraseña") \
    .load()
```

También se pueden crear DataFrames a partir de archivos no estructurados, utilizando un esquema personalizado que define la estructura de los datos. Esto es útil cuando los archivos no contienen metadatos o cuando se requiere validar la estructura de los datos.

Además, Spark SQL permite crear DataFrames a partir de consultas SQL sobre otras tablas o DataFrames existentes:

```
spark.sql("CREATE OR REPLACE TEMP VIEW empleados AS SELECT * FROM tabla_empleados")
df_consulta = spark.sql("SELECT * FROM empleados WHERE edad > 30")
df_consulta.show()
```

6. Métodos de exploración de un DataFrame

Una vez creado un **DataFrame** en Apache Spark, es fundamental poder explorarlo para conocer su estructura, el tipo de datos que contiene y realizar un análisis preliminar antes de aplicar transformaciones o consultas avanzadas. Spark SQL ofrece múltiples métodos para explorar y examinar los DataFrames de manera eficiente.

Uno de los métodos más utilizados es **show()**, que permite visualizar las primeras filas del DataFrame en la consola. Por defecto, muestra las primeras 20 filas, pero es posible especificar un número distinto:

```
df.show(5)
```

Otra función importante es **printSchema()**, que muestra la estructura del DataFrame, incluyendo los nombres de las columnas y los tipos de datos asociados. Esta función es útil para verificar que los datos han sido leídos correctamente y que los tipos son los esperados:

```
df.printSchema()
```

El método **columns** permite obtener una lista con los nombres de las columnas del DataFrame:

```
columnas = df.columns
print(columnas)
```

Para obtener un resumen estadístico de las columnas numéricas, Spark SQL ofrece el método **describe()**. Este método calcula métricas como el conteo, la media, la desviación estándar, el valor mínimo y máximo:

```
df.describe().show()
```

Si se desea obtener información más detallada sobre cada columna, como la cantidad de valores nulos, los tipos de datos y las estadísticas básicas, se puede utilizar el método **summary()**:

```
df.summary().show()
```

El método **count()** permite conocer la cantidad total de registros que contiene el DataFrame:

```
total_registros = df.count()
print(f"Total de registros: {total_registros}")
```

Otra función útil es **head()**, que devuelve la primera fila del DataFrame como un objeto Row. También se puede utilizar **first()**, que cumple la misma función:

```
primera_fila = df.head()
print(primera_fila)
```

Finalmente, para obtener los valores únicos de una columna específica, se puede utilizar el método **distinct()** combinado con **select()**:

```
df.select("columna").distinct().show()
```

Estos métodos de exploración son esenciales para comprender la estructura y el contenido de los datos, facilitando la toma de decisiones informadas sobre las transformaciones y análisis que se realizarán posteriormente.

7. Uso de DataFrames para consultas

Uno de los mayores beneficios de los **DataFrames** en Apache Spark es la posibilidad de realizar consultas sobre los datos de forma eficiente y utilizando un lenguaje familiar como **SQL** o funciones de la API de DataFrame. Esto facilita la exploración, transformación y análisis de grandes volúmenes de datos estructurados.

Una de las formas más directas de consultar un DataFrame es utilizando métodos de filtrado y selección de columnas. Por ejemplo, para seleccionar una columna específica, se puede utilizar el método **select()**:

```
df.select("Nombre").show()
```

También es posible aplicar múltiples transformaciones encadenadas. Por ejemplo, para seleccionar registros donde la edad sea mayor a 30 y ordenar los resultados por nombre:

```
df.filter(df.Edad > 30).orderBy("Nombre").show()
```

Otra funcionalidad poderosa es la posibilidad de registrar un DataFrame como una **vista temporal (Temporary View)** y ejecutar consultas SQL estándar sobre él. Esto se hace con el método **createOrReplaceTempView()**:

```
df.createOrReplaceTempView("empleados")
```

Una vez registrada la vista, es posible ejecutar consultas SQL utilizando el método **spark.sql()**:

```
resultado = spark.sql("SELECT Nombre, Edad FROM empleados WHERE Edad > 30")
resultado.show()
```

Además de consultas simples, se pueden realizar operaciones más complejas como agregaciones, uniones y combinaciones de DataFrames. Por ejemplo, para calcular la edad promedio de los empleados:

```
df.groupBy("Departamento").avg("Edad").show()
```

También es posible unir dos DataFrames utilizando el método `join()`, especificando la columna clave para la unión:

```
df1.join(df2, df1.ID == df2.ID).show()
```

Otra operación común es el uso de expresiones SQL dentro de las funciones de DataFrame mediante `selectExpr()`, que permite aplicar expresiones SQL sin necesidad de registrar una vista:

```
df.selectExpr("Nombre", "Edad + 5 as Edad_Futura").show()
```

8. Optimización de consultas (Catalyst, Tungsten)

Uno de los elementos que diferencian a **Apache Spark** de otras plataformas de procesamiento de datos es su capacidad para optimizar las consultas de forma automática y eficiente. Spark SQL incorpora dos proyectos clave para este propósito: **Catalyst** y **Tungsten**.

El **Catalyst Optimizer** es el motor de optimización de consultas de Spark SQL. Su objetivo principal es transformar las consultas lógicas realizadas por el usuario en planes físicos optimizados que puedan ejecutarse de manera eficiente en el clúster. Catalyst permite a Spark realizar una serie de transformaciones y reescrituras sobre las consultas antes de su ejecución.

El proceso de optimización de Catalyst se divide en varias etapas. La primera es la **análisis lógico**, donde se valida la estructura de la consulta, se resuelven las referencias a columnas y se aplican reglas de simplificación. Posteriormente, en la etapa de **optimización lógica**, se aplican reglas como la eliminación de columnas no utilizadas, el reordenamiento de filtros y la combinación de proyecciones.

A continuación, Catalyst genera un **plan físico**, que define cómo se ejecutará la consulta en el clúster. Este plan incluye detalles sobre las operaciones que se ejecutarán en cada nodo, la forma en que los datos se moverán entre nodos y las particiones que se utilizarán.

Además de Catalyst, Spark SQL incorpora el proyecto **Tungsten**, que se centra en la optimización del uso de recursos a nivel bajo. Tungsten mejora la eficiencia de ejecución mediante técnicas como la gestión eficiente de la memoria, la generación de código bytecode y la ejecución vectorizada.

Uno de los principales beneficios de Tungsten es la **optimización del manejo de memoria**. Tradicionalmente, las plataformas de procesamiento distribuido dependen del recolector de basura de la JVM, lo que puede generar ineficiencias. Tungsten implementa una gestión de memoria fuera de la JVM (off-heap memory), reduciendo la latencia y mejorando la estabilidad.

Otro aspecto destacado de Tungsten es la **generación de código**. En lugar de utilizar estructuras genéricas y dinámicas, Tungsten genera código específico para cada consulta, eliminando sobrecargas y mejorando el rendimiento de ejecución.

La combinación de Catalyst y Tungsten permite que Spark SQL ofrezca un rendimiento comparable e incluso superior al de sistemas tradicionales de bases de datos, mientras mantiene la flexibilidad del procesamiento distribuido.

Estas optimizaciones son transparentes para el usuario. No es necesario escribir código específico para beneficiarse de ellas, ya que se aplican automáticamente cuando se ejecutan consultas sobre DataFrames o vistas temporales.

En conclusión, Catalyst y Tungsten son dos pilares fundamentales que permiten a Spark SQL ejecutar consultas de manera eficiente y escalable, aprovechando al máximo los recursos del clúster y mejorando la experiencia del usuario.

9. Comparación entre DataFrame y Dataset

En **Apache Spark**, los términos **DataFrame** y **Dataset** suelen utilizarse en conjunto, ya que ambos representan estructuras de datos distribuidas y estructuradas. Sin embargo, existen diferencias clave entre ellos que es importante conocer para decidir cuál utilizar en cada escenario.

Un **DataFrame** es una colección distribuida de datos organizados en columnas, similar a una tabla en una base de datos relacional. Esta estructura proporciona una API de alto nivel y permite realizar consultas SQL, transformaciones y agregaciones de forma eficiente y sencilla. En Spark, un DataFrame está disponible en todos los lenguajes soportados: **Python (PySpark)**, **Scala**, **Java** y **R**.

Por otro lado, un **Dataset** es una extensión del DataFrame, introducida en Spark 1.6, y está disponible exclusivamente para los lenguajes **Scala** y **Java**. Un Dataset combina las ventajas de los RDDs y los DataFrames, ofreciendo la seguridad de tipos en tiempo de compilación y un rendimiento optimizado.

Una de las diferencias principales entre DataFrame y Dataset es el **tipado fuerte**. Mientras que los DataFrames son estructuras de datos no tipadas (tipo genérico Row), los Datasets son tipados y permiten definir un esquema específico para los objetos que contienen. Esto facilita la detección de errores en tiempo de compilación y mejora la mantenibilidad del código.

Otra diferencia relevante es el **lenguaje de programación**. En PySpark, la API Dataset no está disponible; el DataFrame es la estructura de datos principal. En cambio, en Scala y Java, los desarrolladores pueden utilizar Datasets cuando desean trabajar con datos tipados.

En cuanto al rendimiento, tanto los DataFrames como los Datasets aprovechan las optimizaciones proporcionadas por el **Catalyst Optimizer** y el proyecto **Tungsten**. Sin embargo, en ciertos casos, los Datasets pueden generar un código más eficiente debido a su tipado fuerte.

A nivel funcional, los DataFrames y Datasets ofrecen operaciones similares, como filtrado, selección, agrupamiento y ordenamiento de datos. No obstante, los Datasets permiten utilizar expresiones funcionales más complejas y aprovechar las ventajas del lenguaje de programación en el que se implementan.

En términos de interoperabilidad, un DataFrame en Spark es simplemente un **Dataset[Row]**, es decir, un Dataset cuya estructura de datos es genérica. Por lo tanto, es posible convertir un DataFrame en un Dataset tipado (en Scala y Java) y viceversa.

10. Funciones definidas por el usuario (UDFs)

En muchos casos, las funciones y operaciones integradas que ofrece Spark SQL pueden no ser suficientes para resolver necesidades específicas del análisis de datos. Para estos escenarios, **Apache Spark** permite la creación de **Funciones Definidas por el Usuario**, conocidas como **UDFs (User Defined Functions)**.

Una **UDF** es una función personalizada que el usuario define para realizar un cálculo o transformación específica sobre los datos de un **DataFrame** o **Dataset**. Estas funciones permiten extender las capacidades de Spark SQL, incorporando lógica que no está disponible en las funciones nativas.

El proceso de creación de una UDF en **PySpark** es sencillo. Primero, se define una función estándar en Python y luego se registra como UDF utilizando el método **udf()**. Por ejemplo:

```
from pyspark.sql.functions import udf
from pyspark.sql.types import StringType

def convertir_mayusculas(nombre):
    return nombre.upper()

udf_mayusculas = udf(convertir_mayusculas, StringType())
```

Una vez registrada la UDF, puede aplicarse sobre un DataFrame utilizando el método `withColumn()`:

```
df = df.withColumn("Nombre_Mayuscula", udf_mayusculas(df["Nombre"]))
df.show()
```

Otra forma de registrar una UDF es como función SQL, permitiendo su uso directo en consultas SQL:

```
spark.udf.register("mayusculas", convertir_mayusculas, StringType())
resultado = spark.sql("SELECT Nombre, mayusculas(Nombre) AS Nombre_Mayuscula FROM empleados")
resultado.show()
```

Es importante tener en cuenta que las UDFs, al estar escritas en un lenguaje externo a la JVM (como Python o R), pueden generar una sobrecarga en el rendimiento. Esto se debe a que Spark debe serializar y deserializar los datos entre la JVM y el lenguaje de la UDF, lo que puede afectar la eficiencia de la ejecución.

Para mitigar este impacto, Spark recomienda, siempre que sea posible, utilizar las funciones integradas de la API de DataFrame o las funciones SQL nativas, ya que estas están optimizadas y no requieren el paso de datos entre entornos.

A partir de Spark 2.3, también es posible utilizar **Pandas UDFs** (también conocidas como UDFs vectorizadas), que permiten procesar los datos en lotes utilizando la librería **Pandas** y mejorar el rendimiento al reducir la sobrecarga de serialización.

En conclusión, las UDFs son una herramienta poderosa para extender las capacidades de Spark SQL y adaptar el procesamiento de datos a necesidades específicas. Sin embargo, deben utilizarse con cuidado para no afectar el rendimiento de las aplicaciones.

11. Fuentes y destinos de datos

Uno de los aspectos que hace a **Apache Spark** una herramienta versátil y poderosa es su capacidad para conectarse a múltiples **fuentes y destinos de datos**, tanto estructurados como semiestructurados. Esta flexibilidad permite integrar información proveniente de diferentes plataformas y sistemas en un mismo flujo de procesamiento y análisis.

Spark SQL admite la lectura y escritura de datos desde y hacia archivos almacenados en sistemas locales, sistemas distribuidos y servicios en la nube. Entre los formatos más comunes soportados se encuentran **CSV, JSON, Parquet, ORC y Avro**. Cada uno de estos formatos presenta ventajas específicas en cuanto a rendimiento, compresión y compatibilidad.

Además de archivos, Spark SQL puede conectarse a bases de datos relacionales mediante el protocolo **JDBC**. Esto permite consultar y escribir datos en sistemas como **MySQL, PostgreSQL, SQL Server, Oracle**, entre otros, facilitando la integración con sistemas empresariales existentes.

En entornos de Big Data, Spark SQL también se conecta a sistemas de almacenamiento distribuido como **HDFS (Hadoop Distributed File System), Amazon S3, Azure Blob Storage y Google Cloud Storage**. Esto permite trabajar con grandes volúmenes de datos almacenados en la nube o en clústeres locales.

Para leer datos desde una fuente, Spark SQL utiliza el método **read**, especificando el formato y la ubicación de los datos. Por ejemplo, para leer un archivo CSV:

```
df = spark.read.csv("ruta/datos.csv", header=True, inferSchema=True)
```

De manera similar, para leer un archivo JSON:

```
df = spark.read.json("ruta/datos.json")
```

Y para leer un archivo Parquet:

```
df = spark.read.parquet("ruta/datos.parquet")
```

En el caso de bases de datos, se puede establecer la conexión utilizando JDBC:


```
df = spark.read \
    .format("jdbc") \
    .option("url", "jdbc:mysql://localhost:3306/basededatos") \
    .option("dbtable", "tabla") \
    .option("user", "usuario") \
    .option("password", "contraseña") \
    .load()
```

Para escribir datos hacia un destino, se utiliza el método **write**, especificando el formato y la ubicación. Por ejemplo, para guardar un DataFrame en formato Parquet:

```
df.write.parquet("ruta/salida.parquet")
```

O para guardar en formato CSV:

```
df.write.csv("ruta/salida.csv", header=True)
```

También es posible guardar los datos en bases de datos relacionales utilizando JDBC:

```
df.write \
    .format("jdbc") \
    .option("url", "jdbc:mysql://localhost:3306/basededatos") \
    .option("dbtable", "nueva_tabla") \
    .option("user", "usuario") \
    .option("password", "contraseña") \
    .save()
```

En resumen, la capacidad de Spark SQL para conectarse a múltiples fuentes y destinos de datos facilita la integración de información y la implementación de flujos de análisis distribuidos, adaptándose a una amplia variedad de escenarios y necesidades.

12. Interoperación de los RDDs, formatos de archivo JSON y Parquet, lectura y escritura desde/hacia distintas fuentes

Uno de los aspectos que distingue a **Apache Spark** es la posibilidad de trabajar con diferentes estructuras de datos y formatos de archivo de manera flexible. En

este sentido, Spark permite la **interoperación entre RDDs y DataFrames**, facilitando la integración de datos estructurados y no estructurados dentro de un mismo flujo de trabajo.

Un **RDD (Resilient Distributed Dataset)** es la estructura de datos más básica en Spark, diseñada para el procesamiento distribuido de datos sin un esquema predefinido. Por otro lado, los **DataFrames** y **Datasets** permiten trabajar con datos estructurados, proporcionando un esquema explícito y ofreciendo un mayor rendimiento gracias a las optimizaciones internas de Spark SQL.

Spark ofrece métodos para convertir un RDD en un DataFrame. Esto es útil cuando los datos provienen de fuentes no estructuradas o cuando se requiere transformar datos complejos. Por ejemplo:

```
rdd = spark.sparkContext.parallelize([("Ana", 25), ("Luis", 30)])
columnas = ["Nombre", "Edad"]
df = spark.createDataFrame(rdd, columnas)
df.show()
```

De manera inversa, también es posible convertir un DataFrame en un RDD utilizando el método **rdd**:

```
rdd_nuevo = df.rdd
print(rdd_nuevo.collect())
```

Además de la interoperación entre RDDs y DataFrames, Spark SQL facilita la lectura y escritura de datos en distintos formatos de archivo, siendo **JSON** y **Parquet** dos de los más utilizados.

El formato **JSON** es ampliamente utilizado para almacenar datos semiestructurados. Spark permite leer archivos JSON de manera sencilla:

```
df_json = spark.read.json("ruta/datos.json")
df_json.show()
```

También es posible guardar un DataFrame en formato JSON:

```
df_json.write.json("ruta/salida.json")
```

El formato **Parquet** es un estándar abierto para el almacenamiento columnar de datos. Ofrece ventajas significativas en términos de compresión y rendimiento, especialmente para consultas analíticas:

```
df_parquet = spark.read.parquet("ruta/datos.parquet")  
df_parquet.show()
```

Y para guardar datos en Parquet:

```
df_parquet.write.parquet("ruta/salida.parquet")
```

Además de JSON y Parquet, Spark soporta otros formatos como **CSV, ORC y Avro**, y permite conectarse a sistemas de almacenamiento distribuido como **HDFS, S3, Azure Blob Storage y Google Cloud Storage**.

La interoperación entre estructuras de datos y formatos de archivo proporciona a los usuarios de Spark una gran flexibilidad para integrar, transformar y analizar datos provenientes de diversas fuentes, adaptándose a las necesidades específicas de cada proyecto.

En conclusión, Spark SQL no solo facilita el procesamiento de datos estructurados, sino que también permite combinar diferentes estructuras y formatos, maximizando la eficiencia y escalabilidad de los flujos de trabajo de análisis de datos.

Cierre

A lo largo de este manual, exploramos en detalle el procesamiento de datos estructurados utilizando **Apache Spark** y su módulo **Spark SQL**. Comprendimos la importancia de trabajar con datos organizados en columnas y filas, y cómo Spark SQL facilita la manipulación, consulta y análisis de estos datos de forma distribuida y escalable.

Estudiamos los elementos fundamentales de Spark SQL, incluyendo la creación y gestión de una **sesión Spark**, la estructura y funcionamiento de los **DataFrames**, y los diferentes métodos para crearlos y explorarlos. También aprendimos a realizar consultas sobre los datos utilizando tanto la API de DataFrame como sentencias SQL estándar.

Además, analizamos los mecanismos de optimización implementados por Spark SQL, como **Catalyst** y **Tungsten**, que permiten ejecutar consultas de forma eficiente. Comprendimos las diferencias entre **DataFrame** y **Dataset**, así como el uso de **Funciones Definidas por el Usuario (UDFs)** para extender las capacidades de procesamiento.

Finalmente, abordamos la interoperación entre RDDs y DataFrames, la lectura y escritura de datos desde y hacia diferentes fuentes, y el manejo de formatos de archivo como **JSON** y **Parquet**.

El conocimiento adquirido en este manual proporciona una base sólida para que los estudiantes y profesionales puedan implementar soluciones de análisis de datos estructurados utilizando Apache Spark, integrando datos de diversas fuentes y aplicando técnicas avanzadas de consulta y transformación.

Referencias

- <https://spark.apache.org/sql/>
- <https://aitor-medrano.github.io/iabd2223/spark/02dataframeAPI.html>
- <https://openwebinars.net/blog/datasets-y-dataframes-en-apache-spark/>
- <https://www.datacamp.com/es/tutorial/pyspark-tutorial-getting-started-with-pyspark>

¡Muchas gracias!

Nos vemos en la próxima lección

